Offline-first, part 1

Local data and the sync engine

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Design for offline

Why offline is a normal state, and why the UI reads local data, never a server response.

Model the local layer

A Drift table with sync metadata, and a live query the UI watches with watch().

Write the sync engine

An outbox, a drain loop, and a delta pull driven by a checkpoint.

Trigger sync sensibly

When to wake the radio, and what background sync can and cannot promise.

PART 1 · OFFLINE-FIRST

The network is not a dependency

It is a background utility that is often, briefly, absent

Offline is a state, not an error

< 1 ms

a local read: no radio, no server, no failure mode

40–200 ms

a good round trip on a warm 4G or 5G link

30 s

the timeout your user waits out before giving up

Worse than no signal is lie-fi: the interface reports a connection, and the request simply hangs until the timeout fires.

An offline-first app has no offline error path. Being offline is not a failure to handle. It is the resting state the UI is built for from the start.

OFFLINE-FIRST

This is not an edge case

Lifts and basements

Concrete and steel block a radio signal completely, for the whole ride.

Planes and trains

Airplane mode is not a bug report, and a tunnel repeats the same loss every few minutes.

Hotel Wi-Fi

DNS resolves, the interface reports a connection, and nothing routes past it.

Hospital wards, rural roads, and festival crowds add the same failure by radio congestion, not by radio absence. All of these are ordinary conditions for real users, not corner cases to special-case away.

The inversion of truth

The same user action, tap Save, under the two architectures.

Online-first: UI = f(server)

1. Tap Save, a spinner appears.
2. The request goes out: latency, retries.
3. The server answers, or it does not.
4. The UI updates, or shows an error.

Offline-first: UI = f(local database)

1. Tap Save, no spinner at all.
2. The write commits locally, in one transaction.
3. watch() re-emits, the UI rebuilds instantly.
4. Sync happens later, in the background.

The network leaves the critical path. It becomes a background job that reconciles two copies of the truth.

OFFLINE-FIRST

CAP, on a device in your pocket

CAP describes what happens during a partition: when the two sides cannot talk, you either refuse to answer, or you answer with data that may be stale. You cannot do both.

Partition tolerance

Not a choice on mobile. A phone in a lift is the resting state, not a rare fault to engineer around.

Availability

You keep it. Every read comes from the local database, and every write is accepted, signal or no signal.

Consistency

You give it up. What you get instead is eventual consistency, and the cost of it is the rest of this lecture and the next one.

The network picks P for you. The only real choice left is whether the app stays useful while partitioned, or freezes waiting for data it cannot reach.

Why the project grades this directly

Requirement 3 of 5

The app must work with no network and sync later, with a stated conflict rule.

Milestone 7

By the seventh project session the app runs offline and syncs. Lab 5 builds exactly this.

Everything from here builds toward Lab 5. ADMD Lecture 9 covered local storage. This lecture adds the sync metadata, the outbox, and the drain loop on top of it.

PART 2 · LOCAL DATABASE

One source of truth, on the device

Schema, reactive reads, and an outbox for every write

LOCAL DATABASE

Firestore’s cache, or your own SQLite

	FIRESTORE OFFLINE CACHE	DRIFT (SQLITE)
Types	dynamic maps, checked at run time	compile-time checked SQL and rows
Reactive UI	snapshots() streams	watch() streams, on any query
Offline queries	full scans, no local indexes	your indexes work offline too
Transactions	online only	full local ACID transactions
Cost	billed per document read	flat: whatever your backend costs

Firestore is a managed sync engine you cannot open. ADMD Lecture 9 introduced sqflite. Drift sits on the same SQLite engine, with generated queries.

A Drift table with sync metadata

```
class Notes extends Table {
  TextColumn get id => text()();       // client-made UUID
  TextColumn get title => text()();
  DateTimeColumn get updatedAt => dateTime()();
  DateTimeColumn get deletedAt => dateTime().nullable()();
  BoolColumn get isDirty =>
      boolean().withDefault(const Constant(true))();
  IntColumn get version =>
      integer().withDefault(const Constant(0))();
  @override
  Set<Column> get primaryKey => {id};
}
```

Four columns make a table syncable: updatedAt, deletedAt, isDirty, version.

Generating the boilerplate

```
dart run build_runner build \
    --delete-conflicting-outputs
```

Drift reads the Notes table and writes the DAO mixin, the generated Note row class, and the companion classes the rest of this lecture uses. Run it again after every schema change.

⚠ Schema versions

Adding a column bumps the schema version. Write a migration step for it, or existing installs crash on the first launch after an update.

What each sync column means

updatedAt

When this device last touched the row. It is the input to any timestamp-based resolution, and its weakness is the subject of Lecture 9.

deletedAt

The tombstone. A delete is a write, never a local DELETE statement, or the server never learns the row is gone.

isDirty

True from the moment the row is touched until the server has acknowledged it. The drain loop clears it.

version

What this device last saw from the server, so the resolver can tell “we are ahead” from “both sides moved”.

The UI reads local data

```
class NotesDao extends DatabaseAccessor<AppDb>
    with _$NotesDaoMixin {
  NotesDao(super.db);

  // Drift re-runs this and re-emits whenever the
  // table changes: from the user, or from sync.
  Stream<List<Note>> watchActive() => (select(notes)
        ..where((n) => n.deletedAt.isNull())
        ..orderBy([(n) => OrderingTerm.desc(n.updatedAt)]))
      .watch();
}
```

One stream, two writers. A user edit and a sync update both flow through this same watch() call.

The outbox table

```
class Outbox extends Table {
  TextColumn get id => text()();
  TextColumn get entity => text()();
  TextColumn get entityId => text()();
  TextColumn get op => text()();        // upsert or delete
  TextColumn get payload => text()();   // JSON, ready to POST
  DateTimeColumn get queuedAt => dateTime()();

  @override
  Set<Column> get primaryKey => {id};
}
```

A durable, ordered list of intents. It survives the process being killed mid-request, which a variable holding a pending request in memory never would.

Writes go to the table and an outbox

```
Future<void> saveNote(Note n) => db.transaction(() async {
  final row = n.copyWith(
      updatedAt: DateTime.now(), isDirty: true);
  await into(notes).insertOnConflictUpdate(row);
  await into(outbox).insert(OutboxCompanion.insert(
    id: uuid.v4(), entity: 'note', entityId: n.id,
    op: 'upsert', payload: jsonEncode(row.toJson()),
  ));
});
```

One transaction, two effects. The row and the intent to sync it commit together, so the outbox can never drift out of step with the data. watch() fires the moment this commits, long before any packet leaves the device.

A delete is a write, not a DELETE

```
Future<void> deleteNote(String id) => db.transaction(() async {
  await (update(notes)..where((n) => n.id.equals(id)))
      .write(NotesCompanion(
          deletedAt: Value(DateTime.now()),
          isDirty: const Value(true)));
  await into(outbox).insert(OutboxCompanion.insert(
    id: uuid.v4(), entity: 'note', entityId: id,
    op: 'delete', payload: '',
  ));
});
```

The row stays in the table with deletedAt set. The watch() query from two slides ago filters it out, so the UI never sees it, but the sync engine still has a row to push.

Why not just call the API directly

The call can fail

A dropped connection loses the write, and nothing records that the local row and the server now disagree.

The app can be killed

Mid-request, the process dies. An in-memory retry queue dies with it; a table on disk does not.

Order matters

Two edits to the same row must reach the server in the order the user made them, not in request-completion order.

The outbox is the durable record of intent. Everything the drain loop does in the next section exists to empty this table, one row at a time, in order.

PART 3 · SYNC ENGINE

Two loops and a checkpoint

Push what this device changed, pull what everyone else changed, remember where you got to

The two loops

Neither loop runs while the user is waiting. Both are triggered by events.

Push: what this device changed

1. Read the outbox, oldest first.
2. POST it, with an idempotency key.
3. The server accepts, assigns a version.
4. Drop the row, clear isDirty.

Pull: what everyone else changed

1. GET /sync since the last checkpoint.
2. The server returns a delta.
3. Upsert, or resolve if isDirty is set.
4. Save the new checkpoint: the server's clock.

The checkpoint is the whole protocol. Store the timestamp the server reported, never the one your device read, or clock skew makes you skip rows.

Draining the outbox

```
Future<void> drainOutbox() async {
  final batch = await (select(outbox)
        ..orderBy([(o) => OrderingTerm.asc(o.queuedAt)])
        ..limit(50)).get();
  for (final item in batch) {
    await api.push(item, idempotencyKey: item.id);
    await db.transaction(() async {
      await (delete(outbox)
          ..where((o) => o.id.equals(item.id))).go();
      await notesDao.clearDirty(item.entityId);
    });
  }
}
```

FIFO, and stop on the first failure

```
try {
  await api.push(item, idempotencyKey: item.id);
} on ApiFailure catch (e) {
  if (e.permanent) { await parkItem(item); continue; }
  break;   // transient: keep order, retry later
}
```

Never skip ahead

Reordering the queue reorders the user's own edits.

Park poison messages

A permanent 4xx fails forever. Move it aside and let the rest through.

Idempotency: safe to push twice

A reply lost on the way back looks identical to a request the server never received. The client cannot tell the difference, so sending the same intent again must be safe.

The key is the outbox row id

Generated once, on the device, when the write happened. It travels unchanged with every retry of that same intent.

The server deduplicates

It remembers keys it has already applied, and returns the same result again instead of applying the write twice.

This is not optional once retries exist. Any network call an app retries needs an idempotency key, on a phone as much as between two backend services.

Delta sync: only what changed

```
Future<void> pull() async {
  var cursor = await meta.checkpoint();  // the server's clock
  while (true) {
    final page = await api.get('/sync', {'since': cursor});
    await db.transaction(() async {
      for (final r in page.rows) {
        await notesDao.upsertFromServer(r);
      }
      await meta.setCheckpoint(page.serverCursor);
    });
    if (!page.hasMore) break;
    cursor = page.serverCursor;
  }
}
```

What one page of the response holds

```
GET /sync?since=2026-08-01T09:14:22Z
{ "serverCursor": "2026-08-01T09:31:05Z",
  "hasMore": false,
  "rows": [
    {"id": "a1", "title": "Milk", "deletedAt": null},
    {"id": "b2", "deletedAt": "2026-08-01T09:20:00Z"}
  ]}
```

Deletes are rows

Absent means not changed, never gone.

Paged

Two weeks offline is not one whole backlog.

One cursor

Moves forward only once its page commits.

Sync, in three numbers

50

rows per outbox batch: enough to drain fast, small enough to retry cheaply

1

transaction per pulled page, so a crash mid-sync leaves a consistent database

15 min

the usual floor for a periodic background task on Android

None of these numbers matters as much as the event that starts the loop. The next two slides are about that trigger.

When to wake the radio

	HOW	LATENCY	RADIO COST	USE IT FOR
Push	server holds a socket open	real time	high: always on	chat, live tracking
Poll	a timer fires either way	half interval	high, mostly wasted	legacy backends
Delta on trigger	sync on resume or reconnect	seconds	low: one per event	almost every app
Nudge and delta	silent push, then a pull	near real time	low: existing channel	feeds, inboxes

Sync on events, not on a timer. App resumed, connectivity returned, or the user asked. A fixed interval wastes battery.

Background sync, and what it gives you

```
void didChangeAppLifecycleState(s) {
  if (s == AppLifecycleState.resumed)
    sync.now();
}

connectivity.onChanged.listen((r) {
  if (r != ConnectivityResult.none)
    sync.now();
});
```

Android: WorkManager.
Roughly a 15-minute floor.
iOS: BGTaskScheduler.
Never guaranteed to run.

Foreground triggers are the contract; background work is a bonus. The syncs you can rely on are the ones you trigger yourself, not the ones the OS schedules.

What Lab 5 asks you to build

A Drift table, with the outbox

Your main entity, with the four sync columns from this lecture, and writes that go to both tables in one transaction.

A drain loop against a tiny server

An in-memory Go server that keeps a version per row. Full source is on a reference slide in the lab.

Delta sync on start

Pull since the last checkpoint, upsert what came back, save the new checkpoint.

Acceptance

Airplane mode on, edit, airplane mode off: the server has the edit. Lecture 9 adds the conflict rule this lab needs next.

RECAP

Three things to remember

1. **The UI reads the local database, never the network.** Sync is a background job that reconciles two copies of the truth after the fact.
2. **Every write is two writes in one transaction.** The row, and an outbox entry recording the intent to sync it.
3. **Sync on events, not on a timer.** App resumed, connectivity returned, or the user asked. Background execution is a bonus, never the contract.

FURTHER READING

drift.simonbinder.eu · Drift: transactions and DAOs · docs.flutter.dev/cookbook/persistence · WorkManager docs

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel